

Random Forest

Definition

A Random Forest is an ensemble machine learning model that combines multiple decision trees. Each tree in the forest is trained on a random sample of the data (bootstrap sampling) and considers only a random subset of features when making splits (feature randomization).

Dataset Used

Training Set



0	1	0	0	85	85	0	No
1	1	0	0	80	90	1	No
2	0	1	0	83	78	0	Yes
3	0	0	1	70	96	0	Yes
4	0	0	1	68	80	0	Yes
5	0	0	1	65	70	1	No
6	0	1	0	64	65	1	Yes
7	1	0	0	72	95	0	No
8	1	0	0	69	70	0	Yes
9	0	0	1	75	10	0	Yes
10	1	0	0	75	70	1	Yes
11	0	1	0	72	90	1	Yes
12	0	1	0	81	75	0	Yes
13	0	0	1	71	80	1	No

Test Set



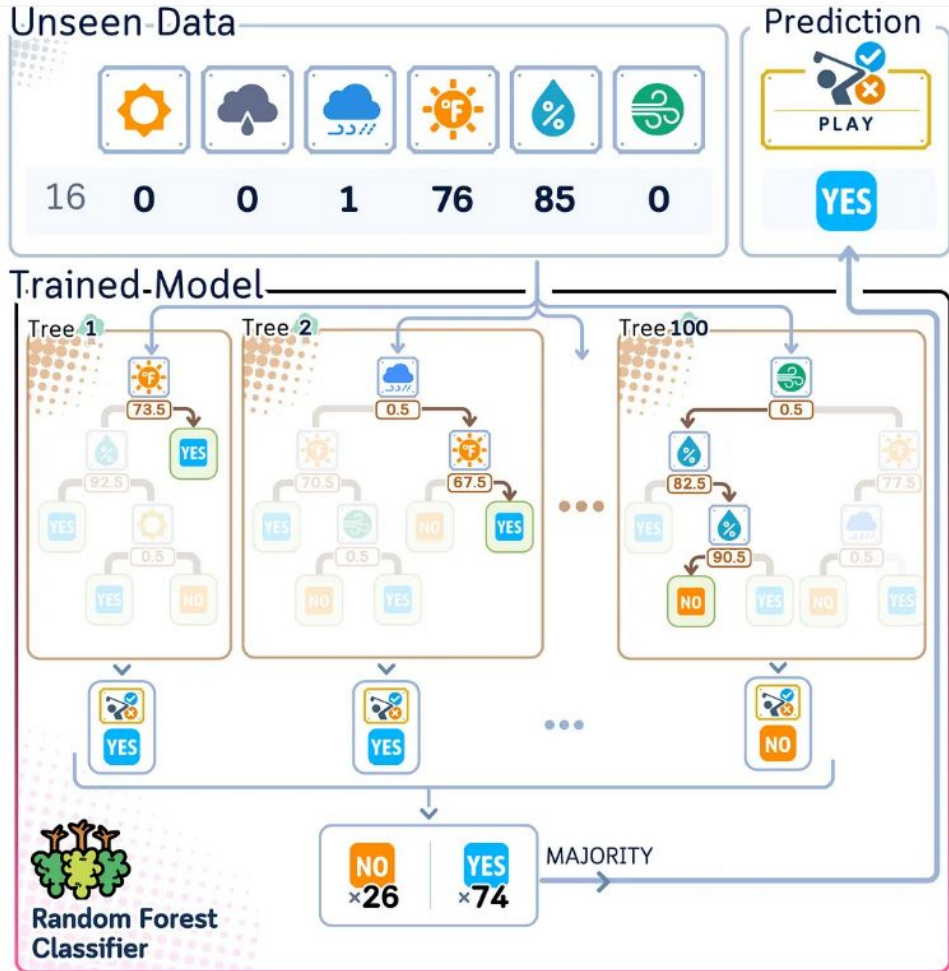
14	1	0	0	81	88	0	No
15	0	0	1	74	92	1	Yes
16	0	1	0	76	85	0	Yes
17	1	0	0	78	75	0	No
18	1	0	0	82	92	0	No
19	0	1	0	67	90	1	No
20	0	0	1	85	85	1	Yes
21	0	1	0	73	88	0	Yes
22	1	0	0	88	65	0	Yes
23	0	0	1	77	70	0	Yes
24	1	0	0	79	60	1	Yes
25	0	0	1	80	95	1	Yes
26	0	1	0	66	70	0	No
27	0	0	1	84	78	1	Yes

Columns: 'Overcast' (one-hot-encoded into 3 columns), 'Temperature' (in Fahrenheit), 'Humidity' (in %), 'Windy' (Yes/No) and 'Play' (Yes/No, target feature)

Main Mechanism

1. **Bootstrap Sampling:** Each tree gets its own unique training set, created by randomly sampling from the original data with replacement. This means some data points may appear multiple times while others aren't used.
2. **Random Feature Selection:** When making a split, each tree only considers a random subset of features (typically square root of total features).
3. **Growing Trees:** Each tree grows using only its bootstrap sample and selected features, making splits until it reaches a stopping point (like pure groups or minimum sample size).
4. **Final Prediction:** All trees vote together for the final prediction. For classification, take the majority vote of class predictions; for regression, average the predicted values from all trees.

Inference



A Random Forest Classifier makes predictions by combining results from 100 different decision trees, each analyzing features like temperature and outlook conditions. The final prediction comes from the most common answer among all trees.

Training Steps

Step 1: Bootstrap Sample Creation

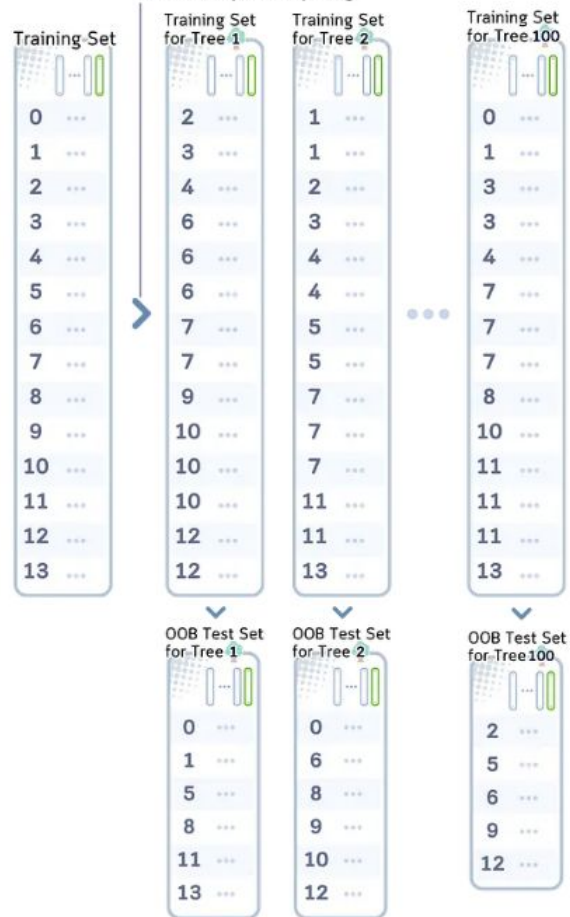
1.0. Set the number of trees (default = 100)

1.1. For each tree in the forest:

a. Create new training set by random sampling original data with replacement until reaching original dataset size. This is called **bootstrap sampling**.

b. Mark and set aside non-selected samples as out-of-bag (OOB) samples for later error estimation

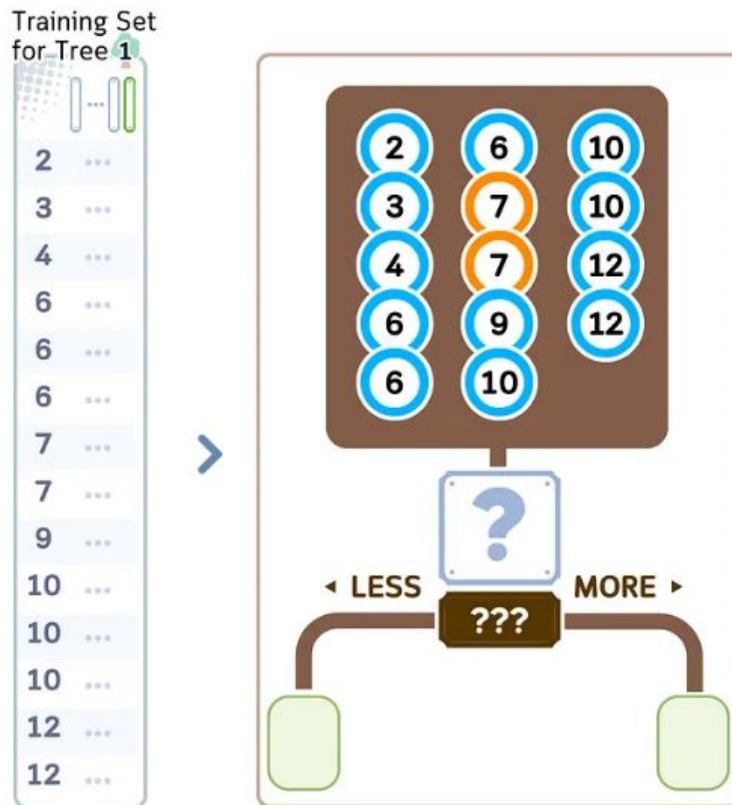
Bootstrap Sampling



Random Forest creates different training sets for each tree by randomly picking data points from the original training set, with some numbers appearing multiple times. The unused data points become test sets for checking each tree's performance.

2.1. Start at root node with complete bootstrap sample

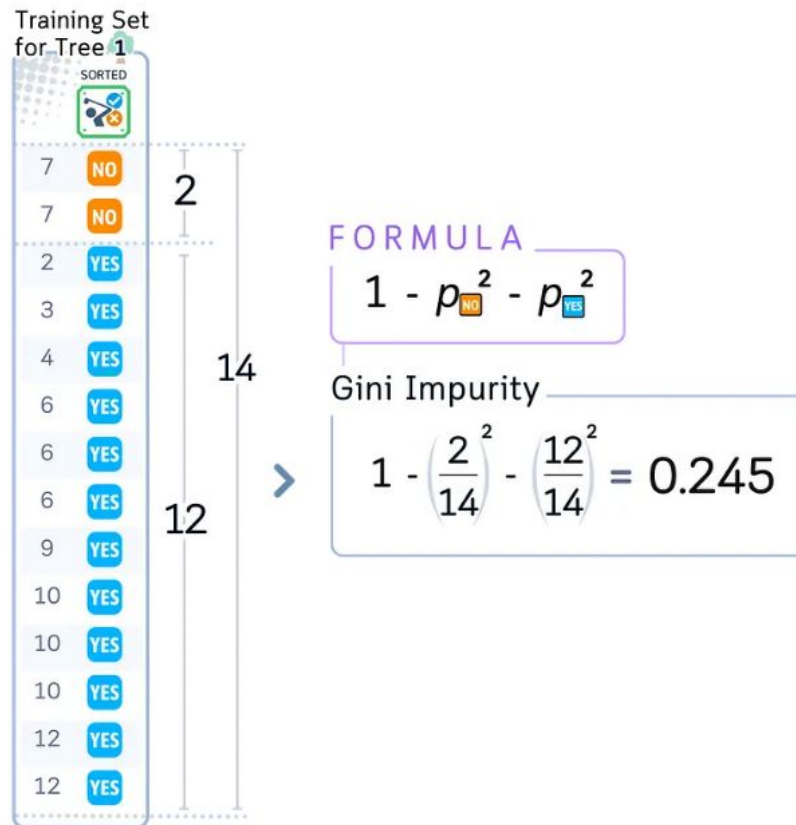
Tree Creation



When building each decision tree, Random Forest considers a subset of data points and creates splits based on questions about their values — sending smaller values to the left and larger values to the right to make predictions.

- a. Calculate initial node impurity using all samples in node
- Classification: Gini or entropy
- Regression: MSE

Parent Impurity



Random Forest starts by calculating the Gini Impurity of the entire dataset (before any splits) using the ratio of YES and NO labels — a measure of how mixed the labels are in the current data.

Feature Selection

b. Select random subset of features from total available features:

- Classification: $\sqrt{n_features}$
- Regression: $n_features/3$

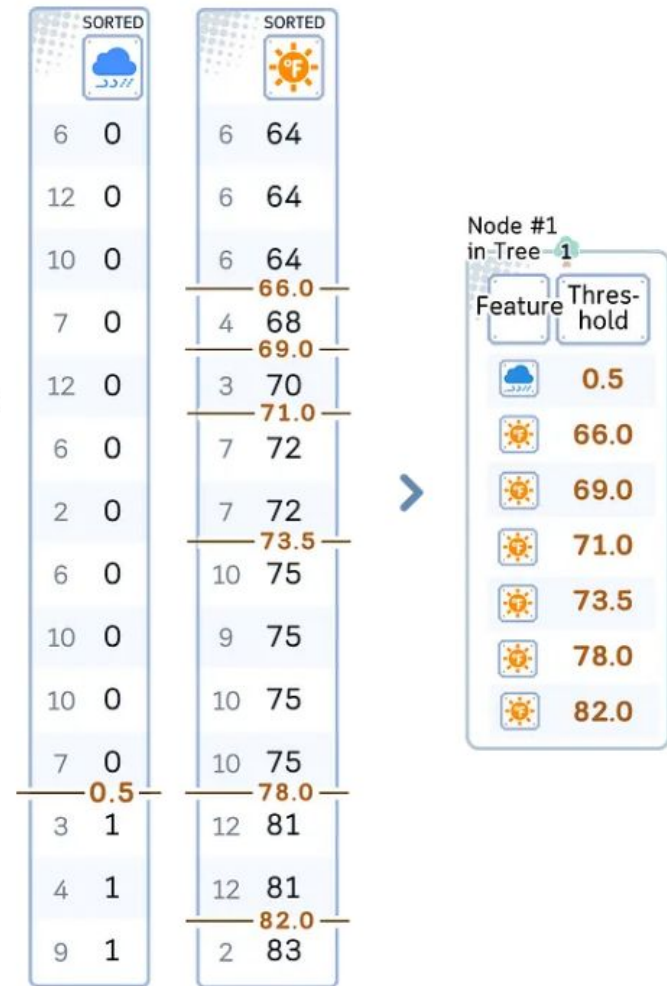


For each split in a tree, Random Forest randomly picks a subset of weather features (here 2 out of 6) to consider, making each tree focus on different aspects of the data.

Purity Check for each feati

c. For each selected feature:

- Sort data points by feature values
- Identify potential split points (midpoints between consecutive unique feature values)



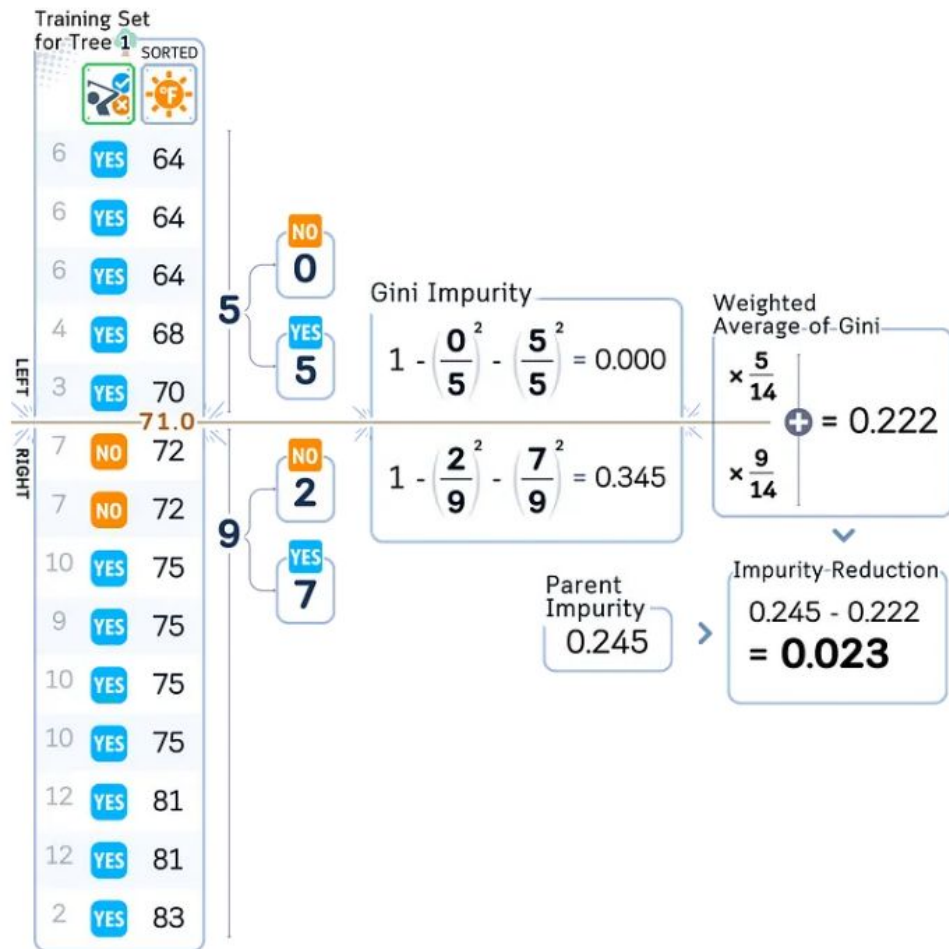
For each chosen feature, Random Forest looks at all possible split points in the sorted data (like temperature values 66.0, 69.0, 71.0, etc.) to find the best way to separate the data into two groups.

Check for each Candidate's Splits

d. For each potential split point:

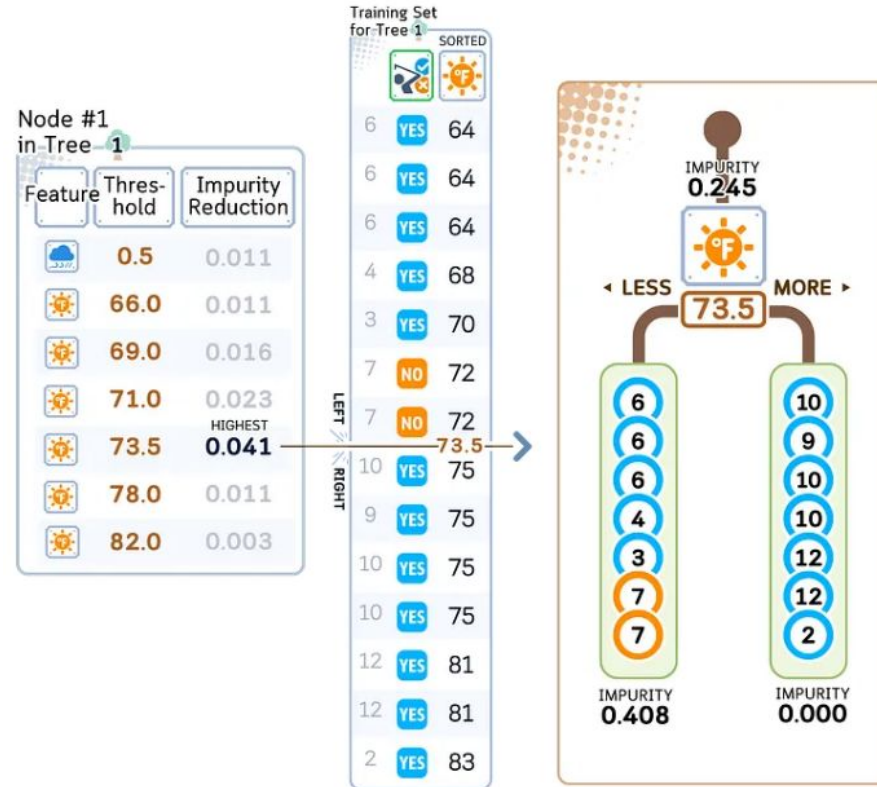
- Divide samples into left and right groups
- Calculate left child impurity using its samples
- Calculate right child impurity using its samples
- Calculate impurity reduction:

parent_impurity — (left_weight × left_impurity + right_weight × right_impurity)



To find the best split point, Random Forest calculates Gini Impurity for each possible split, takes a weighted average based on group sizes, and picks the split that gives the biggest reduction in impurity from the parent node.

e. Split the current node data using the feature and split point that gives the highest impurity reduction. Then pass data points to the respective child nodes.

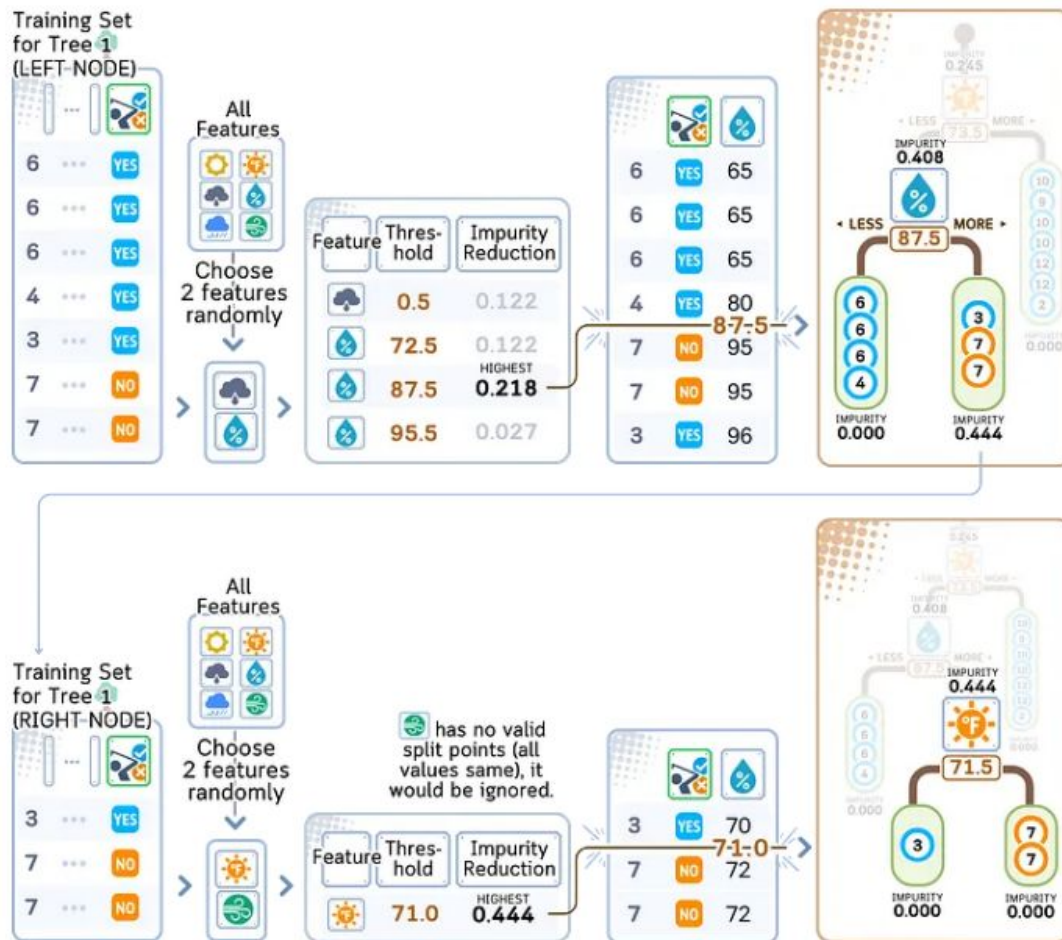


After comparing all possible splits, Random Forest picks the temperature threshold of 73.5°F as it gives the largest impurity reduction (0.041), creating two groups: one mixed group with temperatures below 73.5°F and one pure group.

Loop until

f. For each child node, repeat the process (step b-e) until:

- Pure node or minimum impurity decrease
- Minimum samples threshold
- Maximum depth
- Maximum leaf nodes



This process continues for each new group (node): randomly select features, find the best split point, and divide the data further until each group is pure (all YES or all NO) or can't be split anymore.

For 100 Trees

Repeat the whole Step 2 for other bootstrap samples.

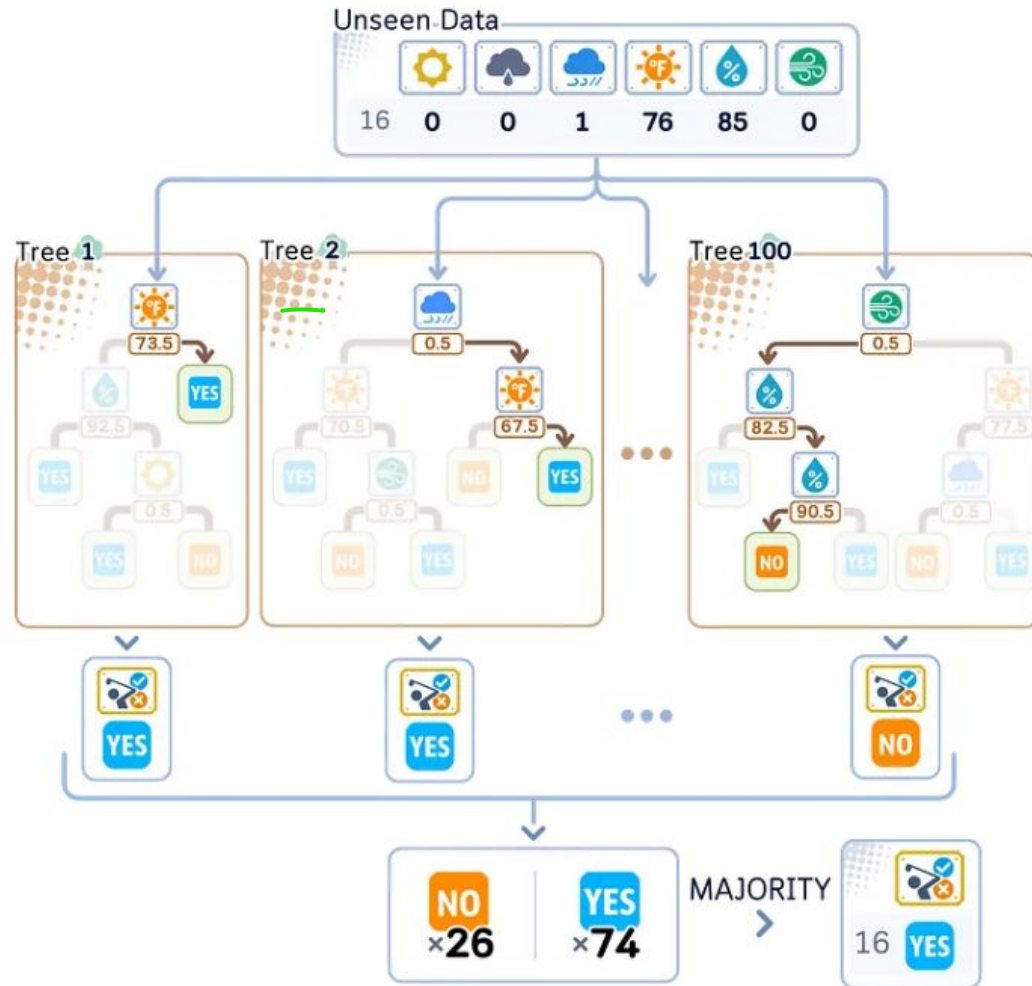


Each decision tree in the Random Forest splits data in different ways using different features and thresholds. This variety helps the forest make better predictions than any single tree.

Inference

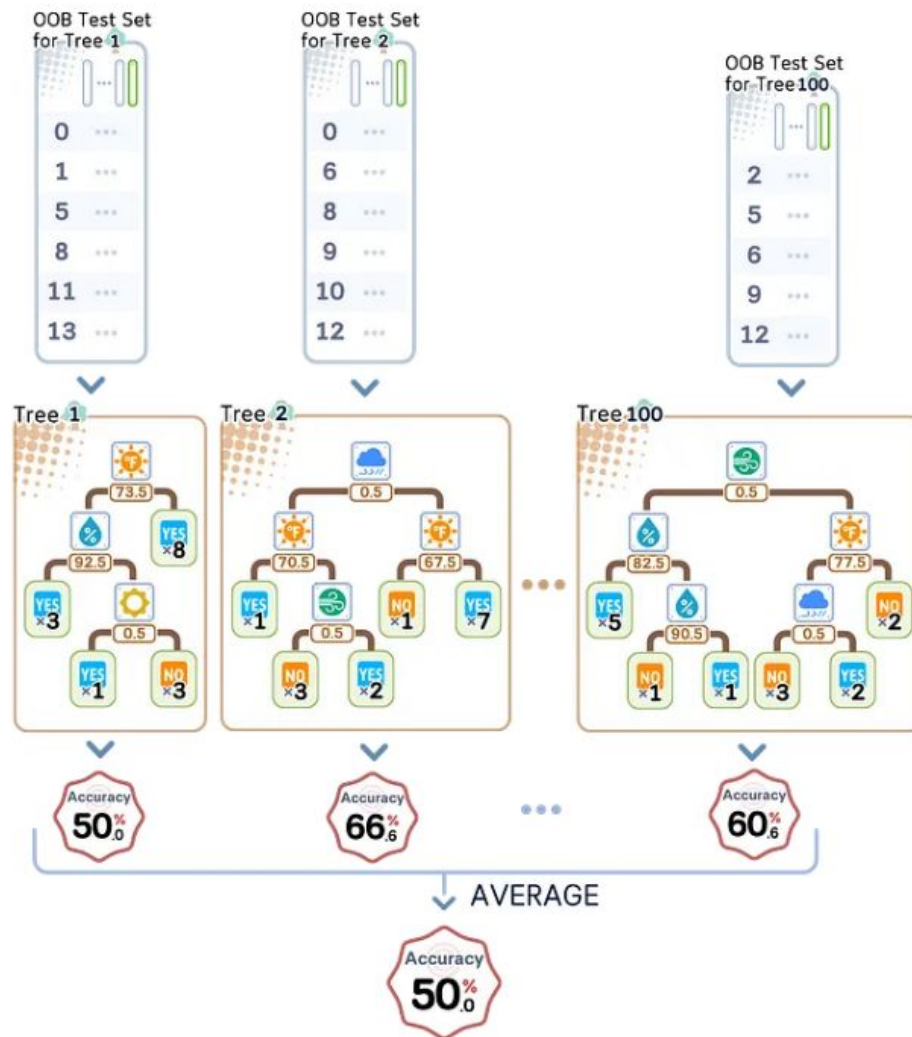
For prediction, route new samples through all trees and aggregate.

- Classification: majority vote
- Regression: mean prediction



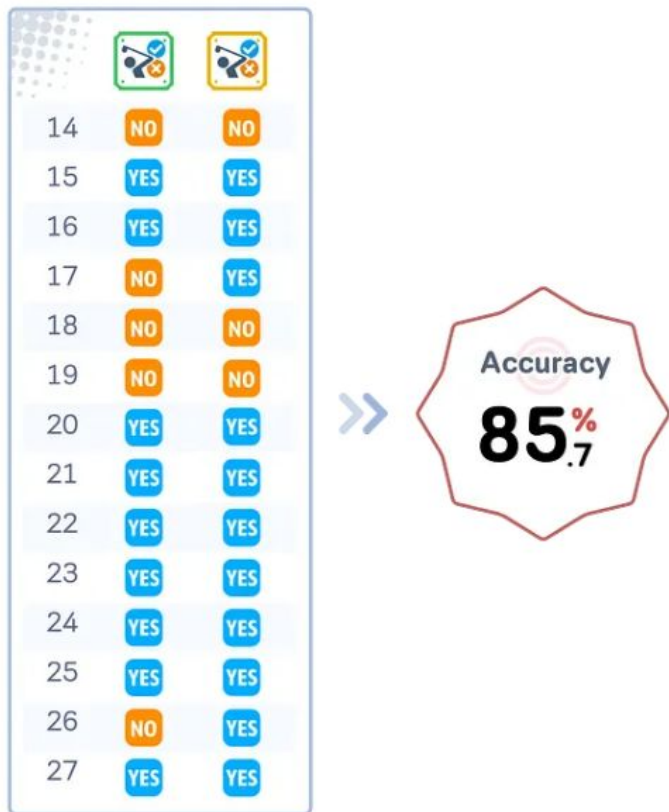
Out-of-Bag (OOB) Evaluation

Remember those samples that didn't get used for training each tree — that leftover 1/3? Those are your OOB samples. Instead of just ignoring them, Random Forest uses them as a convenient validation set for each tree.



Evaluation Step

After building all the trees, we can evaluate the test set.



Pros & Cons

Pros:

1. **Strong and Reliable:** Random Forests give accurate results and are less likely to overfit than single decision trees. By using random sampling and mixing up which features each tree considers at each node, they work well across many problems without needing much adjustment.
2. **Feature Importance:** The model can tell you which features matter most in making predictions by measuring how much each feature helps across all trees. This helps you understand what drives your predictions.
3. **Minimal Preprocessing:** Random Forests handle both numerical and categorical variables well without much preparation. They work well with missing values and outliers, and can find complex relationships in your data automatically.

Pros & Cons

Cons:

1. **Computational Cost:** Training and using the model takes more time as you add more trees or make them deeper. While you can speed up training by using multiple processors, it still needs substantial computing power for big datasets.
2. **Limited Interpretability:** While you can see which features are important overall, it's harder to understand exactly why the model made a specific prediction, unlike with single decision trees. This can be a problem when you need to explain each decision.
3. **Prediction Speed:** To make a prediction, data must go through all trees and then combine their answers. This makes Random Forests slower than simpler models, which might be an issue for real-time applications.